

Agent Infra：与百度王雁鹏聊模型、智能体与 AI 基建 · 学习文档

来源：播客第 81 期《Agent Infra | 与百度王雁鹏聊模型、智能体与 AI 基建》，2026 年 8 月 28 日

时长：约 1 小时 03 分

主持人：狮杰（发言人 2）

嘉宾：王雁鹏，百度智能云 AI 计算首席科学家（发言人 1）

嘉宾背景：2011 年加入百度，国内第一批在互联网公司做硬件的人。入行的时间节点是"第一次有互联网公司采购服务器数量超过 1 万台"。职业生涯主线是针对重要负载做全栈优化——从应用到 infra 到硬件，包括自研 SSD、自研 CPU、硬盘等

本文档定位：按主题重组访谈内容，还原被语音识别打乱的术语，保留完整论证链条

目录

- 1. 什么是 Infra：这一代最大的变化
- 2. Agent 的三个阶段与当前的卡点
- 3. 变与不变：Infra 人如何应对"青春期的孩子"
- 4. Agent 范式解剖：Harness = Loop + Tools + Memory
- 5. Agent 会越来越多还是会收敛？模型即 OS
- 6. Agent 时代对 Infra 三大支柱的冲击
- 7. CPU 消耗量：判断 Agent 是否真落地的领先指标
- 8. Agent Infra 的三大组成部分
- 9. 巨头格局：四种路线的对比
- 10. 垂直整合：所有人都会走，但不一定走得通
- 11. 推理系统：生态里唯一的缝隙
- 12. 中国 Infra 的不同路径：多芯 + 开源
- 13. 百度的打法与新指标 DAA
- 14. 为什么 Token 指标会失效
- 15. 结语：为什么这是工程师最好的时代
- 16. 术语勘误表
- 17. 核心结论速查

1. 什么是 Infra：这一代最大的变化

1.1 Infra 的本质定义

"Infra 其实就是咱们抽象一个层次，这个层次有足够的通用性。往大了讲，今天台积电是不是 infra？它也是 infra，它是做制造的 infra，因为它的影响面足够大。"

更完整的定义（后文补充）：

"你去**抽象**一层，让这一层能够很好地**屏蔽下面的东西**，然后能够更好地**服务上面的**。这一层的概念要对下面有很好的**抽象**，对上面有很好的**支撑**。基于这个层次，我可以做各种不同的应用、做各种开发。"

主持人的通俗类比：

"Infra 就像一个大的交通基建，您就是那个总工程师的角色。"（嘉宾认可）

1.2 Infra 的历史演进

操作系统 (Linux) → 数据库 → 分布式系统 → 云 → AI Infra
└──────── 全部是"纯软件"概念 ─────────┘ └─ 软硬一体 ─┘

为什么上一代 **infra** 全是软件视角：

"因为当时是一个 **CPU** 的时代，芯片已经给你固定好了。**CPU** 本质上就是一个能够跑任何应用的芯片，而且可以很均衡地跑各种应用，你不需要为每个应用去做深度优化。"

1.3 这一代最大的变化：从"纯软件"到"软硬一体"

"这一代 AI infra 有一个巨大的区别：我们是以 **GPU** 为核心构建的 **infra**。

GPU 是什么东西？它极大地释放了算力，但是它牺牲了通用性。原来 CPU 可以做任何事情，但 GPU 你要想把它用好是非常困难的。它相当于把算力给你堆砌得很高，但我在各个场景、各个算法上面能不能发挥这个算力，需要很多很多的工作。

所以它把原来偏软件的 infra，推向了**软硬一体**的 **infra**。这是 AI infra 这一层最大的变化。"

由此带来的人才要求变化：

"做 **AI infra** 的人，一定是从芯片的体系结构熟悉起，然后做上来，而不是原来只是做算法。"

1.4 为什么 Infra 这一轮变得如此重要（关键论证）

"原来说的 IT 更多的是服务互联网的 IT，它其实是一个**成本项**。我服务 1 万个用户和服务 1 亿个用户，成本消耗差别不大，它没有什么**边际成本**。

但在 **AI** 时代，这一轮 **AI** 就是算力的突破，然后才让我们发现了一个通用的算法，所以它天然就跟算力绑定。智能等于算力，所以它天然就变成了你的生产资料。"

"为什么大家一路推导到电力"：

做软件 ← 相对容易，所以做软件的人更多
↓
做模型 ← 好像也没那么难
↓
设计芯片 ← 相对难一点，但也还 OK
↓
流片 ← 更难一点
↓
电力 ← 最难

"越往下走越是重资产。不光是人才，你要投入的资源也不是一天两天能弄起来的。这是大家都在重力投 **infra**、或者对 **infra** 有足够关注的一个本质原因。"

这也解释了为什么中美讨论 AI 差距时，大家会一路"捅到电力"去。

2. Agent 的三个阶段与当前的卡点

2.1 三阶段划分

阶段	形态	状态
第一阶段：Chat	获取信息、收集信息，帮助解决生活中的问题	已充分实现（ChatGPT 是震惊世界的时刻）
第二阶段：Agent 完成任务	真正进入"完成任务"的环节，可以帮助完成真实世界的任务	今年（2026）真正火起来，从年初的 OpenClaw 开始
第三阶段：进入生产环节	更广泛的使用，进入严肃任务的环节	尚未到达

2.2 对 Infra 意味着什么

"完成任务这个环节对 **infra** 来说，意味着我从原来只是去做一个 **API** 的模型推理调用，变成了完成一个真实的任务。

完成一个真实的任务意味着我得有一个完整的环境。今天人操作的所有基础设施、用的各种软件，都要被 **agent**、被 **AI** 去用了。而它的用法当然和人不一样。

那我们的基建怎么做？我们的各种工具怎么搭建才更适合 **agent** 去使用？这就是当前我看到的下一阶段最重要的事情。"

2.3 当前的卡点：Demo 容易，线上系统难

主持人转述行业共识：现在的 **agent** 是可用，但还没有到好用和 safely 用的阶段。

嘉宾的具体描述：

"你拿现在的 **agent** 去做一个 **demo** 很容易，但我要做一个线上系统就很难。

什么叫线上系统？就是能够稳定运行的、能够 **cover** 各种边界条件的、能够上线去运行的、让你放心用的这么一个系统。这个东西你会发现大家还比较少。"

另一个观察角度：

"完成某一部分工作相对容易，但是要做一个某一方面的岗位，这件事情还不行。

这反映了什么？我们的生产环境还不够——现在的能力、或者说大家摸索到的这种范式，还不足以快速开发这些东西。比如我去开发一个替代后端研发工程师的角色，行不行？其实现在还是不太行的。"

当前定位：处于"agent 能完成任务、大家都很笃定这件事一定 OK，但这个能力还没到蓬勃发展的临界点"的阶段。

关于岗位替代（主持人问"业内共识是 agent 可以替代岗位、形成新的组织工作流"）：

"这件事一定会发生。我认为是的。"

2.4 真正的制约因素：模型能力的边界在哪里

"我认为制约是：模型的能力边界在哪里？这件事情是一直在探索的。"

这解释了为什么 agent 创业者都很忐忑——而且这个循环已经发生了好几轮：

轮次	当时的热点	结局
第一波	Prompt 工程 ——给你各种 prompt 模板，帮你做广告片、海报等创意生成。当时估值比较高的一批 AI 公司	模型能力提升一下， prompt 就不那么重要了，这一波可能就没有了
第二波	Context Engineering ——一个复杂任务在同一个上下文搞不定，需要拆到不同上下文、不同角色、不同工作流	正在进行

核心因果：模型的能力边界在哪里，就决定了你最终的那个基建。

2.5 为什么 Infra 层"抽象在哪里"还不确定

"现在这一层在哪里，其实是不确定的。因为你的模型一直在变化，模型有什么能力一直在变化，所以你这个抽象到底抽象成什么样？

新时代的数据库是什么？上一个时代绝大多数应用都构建在数据库这一层上面。那新一层类似数据库的东西，是不是今天的 **memory** 系统？它到底要做成什么样子？其实这个事情都还在一个比较快的变化过程中。

反过来，这个东西在变化，大家的开发范式就在变化。开发范式在变化，这个事情就还没有到能够比较广泛地进入工业系统里去。"

主持人的比喻（本期金句）：

"就像一个青春期的孩子不停地在长身体，你好像没办法为他量身定做一套合身的衣服。"

3. 变与不变：Infra 人如何应对"青春期的孩子"

面对高度不确定的环境，嘉宾的方法论是先做确定性的部分。

3.1 不变的第一层：更便宜的 Token（芯云模一体 + 四层架构）

百度云坚持的理念是"芯（片）-云-模（型）一体"、四层架构协同发展。

推导链条：

更好的模型 → 需要更好的算力
更好的算力 → 需要更好的 infra
目标：让单位 token 的成本更低

"智力 = 算力，算力 = 电力。在这个层面上我是不变的。这就是我们新时代定义的、偏算力的 AI infra。"

3.2 不变的第二层：可运行的 Runtime

"光有算力可能还不够，我还得有一套运行时（runtime），能够在生产环境大规模去用它。要求是：

- Agent 得可控
- Agent 得可信任
- 每一次执行的结果是不是可重复的——这个东西也很重要，是很多 agent 从 demo 到生产时遇到的问题

所以在基础设施层面，我需要构建一套能够快速部署、能够快速开发、能够大规模运行的系统。"

这适合底层的云厂商、或者"芯云模一体"的厂商去思考。

3.3 与 Agent Infra 的关系：延展，而非重构

主持人的问题：新的 agent infra 和上个时代的 AI infra，是并存、打补丁、还是彻底重构？

嘉宾的回答：

"上一个时代 AI infra 的工作，现在变成了一个基座，它的目的就是提供更便宜的 token。在这个基础上，我刚才说的那套运行时环境，是为 agent 设计的一个可运行的系统。所以它应该是一个延展的关系。"

原来的 AI infra 交付的产品界面是：一个训练的集群 + 一个推理的集群。

Agent 时代需要的是：更强的推理集群 + 一套 agent 运行时。

4. Agent 范式解剖：Harness = Loop + Tools + Memory

4.1 拆解 Harness

Harness 拆解为三件事：

1. 主循环（Loop）
2. 它所调用的工具（Tools）
3. 它的 Memory 系统

"这套东西的范式能让大家感到有变化，其实就是因为他把这几个东西相对固定下来了。"

4.2 Loop：为什么是通用的

"**Loop** 就是驱动你这个任务不断循环的这套东西，它是一个相对通用的。

说白了就是：大模型去看我上一步完成了什么结果 → 分析这个结果的问题 → 再去走下一步 → 再 **check** 我有没有完成。

这个东西可以完成几乎 **90%** 的任务，而且不需要太多根据不同场景去泛化的东西。我让大模型去调用工具完成一件事，然后我来切割它的结果，跟我想要的标准做对比，看差距在哪里，然后开启下一层。"

从 **Workflow** 编排到 **Agent Loop** 的范式跃迁：

"最早是 **LangChain**，他做 workflow 编排，那时候听起来也比较 make sense——你要完成一个任务，需要多轮调用，就需要把它编排到一个工作流里去。

但最后大家用那个解决不了复杂任务的问题，就变成 **agent loop** 了。你完成这个 **loop**，不需要那种很精细的 **workflow**。"

4.3 Tools：为什么是文件系统而不是 RAG

"还有一个范式就是他用了非常底层的工具——你会看到他用了文件系统。"

RAG vs 文件系统的对比：

	RAG	文件系统
本质	本质上还是一个数据库	一个笔记本
范式	相对固定：你给他一个关键词，他给你抽取跟这个关键词相关的片段	你想存什么就存什么——可以把你所有的工作流、你今天想干什么、想写什么都写进去
信息丰富度	受限	可以存更丰富的信息
通用性	相对不通用	通用

"这套东西是非常底层的工作范式。如果完成任何工作都可以用这套方式，它就变成了一个通用的系统——我不用为任何一个专用任务去开发一个什么东西。"

4.4 这套范式是 **Claude Code** 带来的

"这套方式我认为是在 **Claude Code** 出现的。

在 **Claude 4.5** 推出之后，包括配套的 **Claude Code**，真的可以完成很多原来程序员的工作——尤其是相对固定一点的，比如前端、写一个网站、做一个功能没那么复杂的 **APP**。我觉得那个时候对程序员圈子是一个很大的震撼。

但这套东西在 **Claude Code** 的视角里，它其实只为 **coding** 服务。"

（早期的 **Manus** 其实也在做这套通用的东西。）

5. OpenClaw 为什么震撼：从 Coding 到全场景

5.1 第一反应：好像没什么特别的

"OpenClaw 出来的时候，第一感觉是他好像没有什么特别的。因为在程序员的世界里，已经被震撼过一波了（Claude 4.5 + Claude Code）。它的整体运行框架还是那一套 **harness** 搭建出来的东西。"

5.2 第二反应：它把范式泛化到了全场景

"后来我觉得它确实是一个很震撼人心的东西，因为他把这套东西泛化到了一个更大的通用场景上面。"

大家用 OpenClaw 就可以感受到：

- 它不是像 **Claude Code** 那样，是程序员的那种冷冰冰的交互接口
- 他有记忆
- 他还可以给你提供情绪价值
- 你可以不知道任何上下文

我今天说我想做一个网页——原来你如果不是程序员，根本不知道从哪里下手。但今天他立马就给我做一个出来。

它的价值在于：它真的让人感觉到 **AI** 是可以像一个人一样完成你所有任务的。"

5.3 它做了什么产品化的工作

主持人的概括："它其实是把 coding 那套产品化。"

嘉宾补充它的三个特质：

1. 有记忆
2. 可以帮你抽象出你要的各种 **skill**，帮你沉淀下来
3. 可以在你没有任何知识储备的时候，帮你做一个你不知道的领域的事情

"如果它具有这些特质，是不是你感觉一个人能完成的事儿他都能完成？"

主持人补充的一个维度——人机关系的进化：

"因为他的角色化和拟人化，我可能对他会有更进一步的信任。"

嘉宾认可：

"你可以交给他所有权限的这些东西，你可以放心他去做更多的事情。他是把它从 **coding** 这个场景推向了一个全场景。"

5.4 后续产品的进一步演进：自动提炼 Skill

主持人提到 OpenClaw 之后又火了一波的另一个产品（原稿音译为"Hermas"，具体产品名存疑）。

嘉宾的评价：

-

"我个人觉得范式上好像没有那么大的变化，但他确实做了一点非常好的事情：他可以去自动提炼你的 **skill**，这一点确实是个痛点。因为你的 **skill** 太多了，你也不知道哪个好哪个不好。"

"越用越聪明"在技术层面怎么做到：

"他就会定期地去反思。假设你今天这一天，他可以定期去抽取：你让他完成了哪些任务？这个任务你第一次哪里不好？你给他的反馈是什么？他有没有把它做得更好？他把这些东西整理起来，就会形成一个更好的、明天你可以用的 **skill**。"

主持人的质疑：我跟 Claude / Codex 交互时也可以给反馈，让它下一次进步，这两个好像是一样的？

嘉宾的回应（区分点在于"自动化"和"组件化"）：

"你说的对。所以这就是差别所在——它没有帮你做一个更拟人化的 **memory**，没有自动化地去收集你的 **skill**，没有自动化地去帮你整理知识。它没有帮你做成这些一个个的 **general** 的组件。

所以（这类产品）是在产品化的某一方面帮你解决了一个比较重大的问题，让你觉得它更加好用了。"

6. Agent 会越来越多还是会收敛？模型即 OS

6.1 主持人的问题

Agent 到底是会永恒地越来越多，还是会边际递减——到一个范式收敛的阶段后，agent 又会约束控制它的"人口"，让它在一个最高效的数量里去协作？

6.2 嘉宾的判断：会越来越多，而且是数量级的提升

"我个人觉得会是 **agent** 越来越多。

现在大家看到的这种无序，我认为更多的是大家在探讨摸索 **agent** 的边界，所以有不同的开发方式。

但往后看，如果 agent 的能力边界相对 OK 了，现在的大模型 + **agent** 这一套以后应该是一个类似今天操作系统的东西——它给你提供了一个很好的 agent 开发平台。在这个上面会长出各种不同的场景和应用，这个效率会比原来高很多，非常可能是数量级的提升。"

6.3 核心论据：模型是 OS，但不会吸收一切

"我认为模型以后会是 **OS**。他是一个大脑，他是一个 **OS**，他是一个很聪明的大脑，他的 **IQ** 很高。

但是你想要完成一个任务，你就得有：

- 每一个任务的上下文
- 每一个任务的记忆
- 每一个任务的工作流
- 你做每一项任务踩的坑在哪里、你的专业的门道在哪里

这套东西我认为不会被大模型完全吸收进去。

大模型吸收的是：你思考的方式、思考的方法、你工作的范式。

但每一项具体工作的 **know-how**、**workflow**、记忆能力、积累下来的数据——这些东西我觉得都会是不一样的。”

推论：基于这个判断，就会有很多 **agent** 出现，替代各种不同的工作，它们会有各种组合，最终到生产力真正井喷的结果。

6.4 主持人的天才类比

“通用大脑就跟我们一个天才一样。我们假定天才可以做任何事，但是天才不会从 **day one** 就能够做任何事。当中有一个学习成本是无论如何都要跨越的。我理解那个学习成本，就是在模型之上、这个通用大脑之上，应用可以做的事情。”

嘉宾确认：

“对于你每一项具体工作，你的一二三四步到底是什么样子、怎么样能让交付的结果更好——这些东西我认为不能完全沉淀到模型里面去。”

7. Agent 时代对 Infra 三大支柱的冲击

Infra 三大支柱：算力、网络、存储。做系统本质上就是在平衡这三者。

7.0 最核心的冲击：上下文长度爆炸

“**Agent** 时代最重要的变化是：我的上下文特别长。

原来 chat 时代上下文是比较短的，每次输入的东西比较少。在 agent 时代它的上下文会非常长——所以你看现在最先进的模型已经支持 **1M** 的上下文了。”

主持人的对比：

“过去用 chat 的形式跟模型交互，无非就是你跟他聊多聊少，但再怎么聊，本质上还是一个聊天窗口的量。但你要让 **agent** 帮你执行任务，它过程当中会浏览大量的网页、外挂信息，它自己要调用各种各样的工具——这个消耗的 **token** 比你跟他聊天是多得多的。”

嘉宾补充“小步快跑”的机制：

“执行完了之后，我会把所有的上下文再拿回来再给你，意味着它的输入就会非常长。所以它其实是小步快跑，完成一个任务要迭代很多很多轮。

而且给一轮反馈是大模型自己反馈的——大模型自己再来看你反馈是啥，我再来决定下一步往哪里走。”

所以 **agent** 对 **AI infra** 的第一个冲击就是：上下文会扩充得特别长。这对系统的冲击很大。

7.1 算力：多元芯片是国内的必答题

“对于我们来说，我们就需要支持更多的、更多元化的芯片。”

为什么 **NVIDIA** 强：

"芯片的设计是一个非常系统化的工程，里面有很多细节的取舍。这也是为什么 NVIDIA 今天这么强大的原因——它构建了一整套的算力系统，让现在最先进的模型都在它的这个算力系统上面跑出来。

那我其他家构建的芯片多多少少都会有区别，这就意味着它在模型的调用、推理这些表现上面会有差别，包括网络的配比、存储的配比也会有差别。"

国内外的差异：

	国外	国内
芯片生态	更统一，绝大多数在 NVIDIA 生态上	要面临更多的多芯局面
Infra 难度	相对好一点（在一家技术栈上做）	要更海纳百川

"我们可能还要思考在不同地域的算力，我能不能把它用起来。这些东西也会考虑——你要面临更多的芯片、更多的地域、更多的异构环境。"

7.2 存储：一个滞后了两年的周期性行业

需求侧：

"AI 从 GPU 开始就用了大量的存储芯片。我们需要存更多的上下文，就要用更多的内存。存储的需求量增加，尤其是高端存储——比如搭配 GPU 用的 HBM，其实都是比较短缺的。"

供给侧的三重困难：

困难	说明
扩产周期长	投一个存储产线到能生产出来，至少两年时间过去了
资本开支大	投一个存储产线消耗的钱非常多
强周期性 + 预期错判	见下

为什么大家没预料到：

"前面好些年存储的需求一直往下走，价格一直往下走。因为存储最大的驱动原来是消费电子——手机、电脑。但手机电脑的增长曲线已经比较平稳了，所以整个存储产能其实一直是供过于求的。

所以大家没有预料到这一轮 AI 会这么强劲。"

这就是现在存储涨价的原因：现在的产能是几年前备好的，但这一波 AI 井喷对存储和算力的需求都爆炸了。

存储的挑战要放大到芯片级别看：

"存储这一块挑战，要把视角稍微放大一点，应该一下子到芯片级别——不光是现在看到的 SSD、内存这些东西，大家也会为推理去设计新的芯片。

它是一个整个推理系统的变化，而且这个变化会非常深刻——从芯片设计本身，到系统的设计，到软件的设计。"

7.3 网络：Agent 反而没带来太大变化

主持人的预设：multi-agent 要让海量 agent 高效协作，对网络的挑战是什么？

嘉宾的回答（一个反直觉的判断）：

"我觉得反而 agent 这个东西对网络没有太大的变化。"

网络现在的核心追求是超节点：

什么是超节点？

"原来我的一台服务器基本上就是 8 块 GPU，可以说是物理限制的。在这 8 个 GPU 里面，会有一个快速的高速互联把它们连起来。

但现在我发现 8 块就不够了。所以我希望把原来机内的高速总线扩展到一个更大的规模——从物理上可能扩展到一个机柜、甚至一个机房，都有一个超高速的互联。

它的核心是把原来片内的总线扩展到集群的总线上面，达到的效果是通信带宽提升十倍或者几十倍这么一个数量级的变化。"

"但超节点这个东西是一个趋势，有没有 agent 我认为大家都会做——或者说我们现在不都已经做了吗？单纯讲 agent，我现在还没有看到它带来连接范式的变化。"

8. CPU 消耗量：判断 Agent 是否真落地的领先指标

这是本期访谈最具操作性的一个洞察。

8.1 逻辑推导

"今年大家看到的一个更大的变局是：原来我只是 GPU（在增长），但如果 agent 真的能够蓬勃发展的话，其实你 CPU 的消耗也会变。

为什么？因为原来 CPU 搭的系统是给人用的，现在 agent 的调用会比人更频繁。这意味着你原来给人用的工具，现在要改造成给 agent 用。"

Agent 执行任务的完整循环：

用 GPU 思考一下

↓

思考完了交给 CPU 去做（调用工具、调用搜索 API 等）

↓

CPU 把结果告诉 GPU

↓

再交给 CPU 去做 循环

而且可以并行做 multi-agent：并行 1000 个，每一个都要去调工具

↑

这些全都是 CPU 的消耗

8.2 结论：CPU 消耗量是 Agent 落地的强关联指标

"接下来如果 CPU 的消耗量大量增加，就证明 agent 在生产环境有了更高速的落地。"

主持人的总结：

"CPU 的消耗量，你可以窥见 agent 到底有没有在做好它的工作。"

嘉宾：

"是不是真的在干事儿，而不要老思考。"（用工具的指标跟 CPU 强关联。）

佐证：NVIDIA 今年 GTC 也推出了自己设计的 CPU——正是因为"agent 时代真的要到来的话，CPU 也会更多"。

9. Agent Infra 的三大组成部分

嘉宾对 Agent Infra 的拆分框架。

9.1 运行时环境 (Runtime / Sandbox)

要求：

- 能够快速启停
- 有更好的安全边界

9.2 编排系统 (Orchestration)

"我们有记忆系统、有沙箱的执行系统、还有大模型的思考系统。这些东西怎么样部署到我的环境上面去，能够让它稳定、高效、可复现地运行——这是编排系统的事情。"

9.3 记忆系统 (Memory)

"记忆系统肯定也是一个很关键的事情。我记什么东西？我怎么样去抽取我要的东西回来？"

但这一层的问题在于：开发范式还没定

"今天的 memory 系统是不是上一个时代的数据库？我认为至少今天我还没看到——我要做成什么样的 memory 能够承担这个职责？这一层到底存不存在？"

我觉得这个都还在变化，大家还在一个探索期。"

9.4 【方法论】做应用的人 vs 做 Infra 的人

主持人的观察：当底层技术还在变化、且没到天花板的时候，所有支撑系统的人都要非常灵活，可能每隔很短一段时间就得重新 review 一遍过去的战略。

嘉宾的区分：

做应用的人	做 Infra 的人
该想什么 找到一个痛点，快速做一个产品，让产品满足一部分人的需要	想这一层抽象是不是能够涵盖更多的需求，能不能更好地支撑他们去做各种开发

"尤其在我们今天的时代，我觉得已经变了好几轮了。"（如从 LangChain 的 workflow 编排 → agent loop。）

10. 巨头格局：四种路线的对比

10.0 共同的方向：OS 层与入口之争

"我认为以模型为核心、包括 agent infra 这些东西加在一起，它会形成一个 OS 层。这个操作系统足够庞大，它会形成一个新的生态。在这个生态下面，我就有机会往下去做整合。所以今天大家第一目标就会去抢这个入口，OS 是大家要争夺的焦点。"

什么是"入口"：

"大家要基于我这个东西去开发应用，而且开发完了能在我的这个系统上面高效地跑。这是我认为大家现在要争夺的入口级别的东西。"

能争夺入口的还是大厂吗？—— "我认为往后看可能还是这样，但是每一家的做法不一样。"

10.1 四家的路线对比

	Google	NVIDIA	OpenAI	Anthropic
定位	全栈垂直整合派	从芯片长成的系统公司	生态平台型	聚焦模型 + OS
核心资产	TPU（做了近十年） + JAX + 自研 infra + 模型	芯片 + 互联 + 网络 + CPU + 超节点整机柜 + CUDA + 算子	生态 + 算力自持	Claude Code / harness 这一套
打法	每一层都很强，没有 短板	打破软硬件边界，构建完 整算力系统	抢入口、做生态；与博通 合作做芯片；自建算力 (Stargate)	不造芯片，与各大 云签协议保障算力 供给
资产轻重	重	重	重	相对轻资产

10.2 Google：多年布局在这一代终于 work

"为什么大家觉得今天的 Google 做得比较好？尤其 **Gemini 3** 出来之后——出来之前大家都觉得他有点掉队了，但 Gemini 3 出来之后大家就觉得他一下子又行了。

很大一个原因是：

- TPU 已经做了快十年
- 软件栈：他是通过 JAX 去做的，他也没有复用 PyTorch 这一套东西
- 整个 infra 这一套东西也是他自己的
- 模型这一套东西也是他自己的

所以他从"芯-云-模"这个角度真的是一体化的。而且因为自研了芯片，理论上成本也会更低一些。"

"他感觉内功比较深厚，每一层好像都没有短板。既然没有短板，你就没有理由怀疑他模型一定做不好。"

关于"Codex 追上来了，Google 又不行了"的声音：

"我个人觉得这跟 Gemini 3 出来之前是一样的。我还是觉得他有这么深厚的全栈能力积累，他没有能力做不好。

好比说 Anthropic 成功很大程度上是因为他两年前确定了'我就做 coding 这一件事'。但对于 Google，既然我有这么强的积累和底座能力，我可能会同时支持我做好几个，我可以稍微比你慢一点。我倾向于这种认知。"

10.3 NVIDIA：从被摩尔定律锁死，到打破软硬件边界

先讲被锁死的时代：

"CPU 的时代其实是被 Intel 那一套东西锁死的。你听到的都是摩尔定律。

摩尔定律背后是什么？就是在 CPU 这一层做了一个非常好的切分：我只用把 CPU 的晶体管越堆越多，CPU 的性能就能提升，上面所有的应用是不用改的，性能就自然会提升。

所以大家只会听到摩尔定律，摩尔定律就驱动了整个 IT 世界的发展。它其实是在这一层锁死了。"

NVIDIA 做了什么：

"NVIDIA 从第一天就在不断试图去打破这个边界，而且他赌对了 AI 这条路。

所以你会发现 NVIDIA 今天已经不是一个芯片公司，它是一个系统公司：芯片、互联、网络、CPU、超节点整机柜、CUDA、各种算子、针对各种模型的优化——这些东西都是很难做的。"

打破边界的直接体现：

"CPU 时代，我换一个 CPU，上面软件是不用改的。但你现在换每一代 GPU，它上面的软件都大改了——非常多。这个东西当然 NVIDIA 帮你做了很多。"

垂直整合的极致形态：

"它是从芯片做到了系统、做到了互联、做到了存储、做到了 CPU、做到了软件。这是 NVIDIA 今天这么强大的原因。

你的模型第一天就是在他的这个系统上做出来的。或者说，你最先进的模型是在他给你提供的最先进的系统上面做出来的。由不得你去定义系统——他给你的系统，你才能做出最先进的模型。"

10.4 OpenAI：从骨子里就想做生态型企业

"你看它的打法，它一上来的各种策略，还是一个生态平台型的打法。或者说大家说他更想成为 Google，他是想做一个生态型的企业，我认为他从骨子里一开始就想做这个。

你看他之前推出的 GPT Store，就是想对标 App Store。他的发力点是希望把所有人拉进来，在我这个生态上面形成粘性，自然生态的力量就发挥出来了。"

这也解释了他对算力的态度：

"在这个逻辑上你就可以理解他为什么：

- 一方面跟博通这种去合作做芯片、去投芯片
- 另一方面这个生产资料我得把握在自己手里"

10.5 Anthropic：聚焦高价值任务 + 轻资产合作

"Anthropic 现在的重心更多在想把模型做好，把模型的这个 OS 系统做好。

他现在的 Claude Code 这一套东西，我认为确实是领先的，大家都在追赶他——Codex 也好、什么也好，都在追赶它。他在这块发力比较明显，他希望把这个入口做得更加强大。

Coding 就是它最强大的入口，Coding 就是高价值任务的一个底层的能力。"

一个重要的行业指标转向：

"OpenAI 之前追求的是 DAU 这些东西，其实是一个偏用户侧产品的指标。但他们现在也转向追求 token 消耗——本质上也是为了往价值结果去转。"

Anthropic 的算力策略：

"至于下面，他也知道生产资料很重要，所以他采取的策略就是跟各大厂签订协议——让各个云（亚马逊、Google）.....我也可以买 TPU，亚马逊的芯片我也可以买，我来保证我的算力供给是安全的。当然这里面它自然也要建立把各种芯片用起来的能力。"

为什么 Anthropic 敢轻资产、OpenAI 却把算力当生死攸关的事：

主持人观察：Anthropic 是这些公司里相对轻资产的——没有投资建厂，没有 Stargate 那样的项目，通过相对轻资产的合作方式建立生态。

嘉宾的猜测：

"市面上还有几家云公司可以供给他算力，这个东西短期内对他来说也不是生死攸关的，那我就更聚焦做我现在更想做的事儿。"

11. 垂直整合：所有人都会走，但不一定走得通

11.1 垂直整合的逻辑很对

"我今天知道我的模型往哪里走，我就能够定义一颗更好的芯片。这个当然从逻辑上来说是很对的事情。"

11.2 但实际操作有巨大的不确定性

"我做一颗芯片要两年半以后才能出来。

我今天用模型定义了这颗芯片，这颗芯片两年半出来以后，我还是不是这个模型的需求？很大程度上是不是的。

所以我认为大家都会想去走垂直整合这条路，但实际上你走的时候，你怎么能保证这件整合是成功的？这件事情大家都会尝试去做，但真的能不能做好，其实还有很大的不确定性。这也是每家禀赋不一样带来的。"

11.3 软件公司做芯片的天然 gap

"软件世界和硬件世界是有 gap 的。你是一个软件公司，你想去做好芯片，说实话挑战还挺大的。

它的工作模式不一样，你投入的点、你关注的点、它的周期都不一样。"

11.4 【重要区分】"为自己做的 infra" vs "为所有人做的 infra"

主持人的问题：NVIDIA 作为芯片厂商去做 infra，和 Google 全栈去做 infra，本质区别是什么？

嘉宾的回答（本节最关键的洞察）：

"Google 的整套东西其实是为 Google 自己服务的。如果今天你是一个创业者去用 TPU，我相信是不那么好用的——因为它缺乏软件生态。"

算力可能有 50% 被浪费了，而这个 50% 我觉得都是低估了的。如果你的算子写得不好、系统做得不好，你可能 80% 的算力都发挥不出来。因为他不是为你做的，他只是为他自己做的。但 NVIDIA 今天是有所有最先进的模型都基于他做的，所以这套东西自然就是相对完备的，你在上面就可以组合出很多创新的模式。"

推论：训练系统是一个很难打破 NVIDIA 生态的领域。即使 Google 今天做得这么好，你把 TPU 给别人去做训练，还是一件很困难的事儿。

11.5 关于挑战者的"鸡生蛋"困境

主持人：现在大量需求只能被迫接受 NVIDIA 给的产品，但它并不一定是最优解。所以有很多创业公司、包括博通，在围绕缝隙做新方向。

嘉宾：

"我个人觉得这个还是挺难的。你是在他给你做的这套系统上面做出来的最先进的模型。这个最先进的模型而且还在不断迭代，他还在不断推出新的东西——你怎么能保证你做出来的东西比他的还要好呢？"

这是个有鸡有蛋的问题。今天最先进的模型，是最先进的硬件系统去催生出来的。"

12. 推理系统：生态里唯一的缝隙

"但是你说这个事情有没有缝隙？我认为有的。第一个大的点就在于推理系统。"

12.1 为什么推理是机会

"推理相对来说，我是模型已经训练出来了。那这个时候我可能就不一定完全要用他那套东西了——我就有时间，我再去做一个 for 推理的芯片。"

所以我认为这是一个大的机会。包括 NVIDIA 自己——你看他为什么收购了 Groq？他也是希望在推理上去做更大的变革。"

我认为推理这个东西，是有可能做出不一样的玩家或者不一样的巨头的。"

12.2 从 GTC 看到的两个方向

主持人：所有做 infra 的人一定会 follow 英伟达三月份的 GTC。

嘉宾看到的两个重点：

1. 推理系统——"你会发现今年 NVIDIA 讲得最多的也是推理系统。他推出了一个 GPU 和它收购的 Groq 的 LPU 的混合推理方案，这是他现在投入的重点。"
2. CPU——"agent 时代真的要到来的话，CPU 也会更多。所以他今年也推了他自己设计的 CPU。"

12.3 值得关注的创业方向

方向	代表	说明
推理芯片	以 Groq 为代表（已被 NVIDIA 收购）	"它只代表了一个技术方向，在这个技术方向上其实有各种不同的尝试"
Infra 效率	Fireworks 等	"怎么样让 infra 的效率提升，把每一滴算力的性能都发挥出来.....怎么样能把各种芯片都发挥好，降低最终呈现的 token 成本"
Agent 记忆存储	主持人提到的一家做 agent 时代记忆存储基建的公司	"为了让 agent 以更快的速度、更低的成本、更灵活地找到所需要的记忆"

嘉宾对"把芯片用起来"的强调："你要想把这芯片用起来，这件事儿其实还是很困难的。这个上面还是有很多的功夫可以做。"

13. 中国 Infra 的不同路径：多芯 + 开源

13.1 两个差异

维度	国外	国内
芯片环境	绝大多数在 NVIDIA 生态上，更统一	天然要考虑国产芯片，要做更多的多芯适配
开源生态	相对封闭	"现在中国的开源是远远领先于美国的"

13.2 开源为什么是中国 Infra 的结构性优势

"开源带来的好处就是大家真的能够共同进步。

你今天其实不知道国外那几家闭源的模型结构是什么样子——那我的芯片到底怎么去设计呢？其实我是不知道的。

但国内开源生态很蓬勃。现在领先的那几个模型、反响最好的几个模型其实都是开源的。所以我就知道它的架构往哪里走、怎么去变化——其实是更透明的。"

这个透明度对谁重要：

- 芯片设计公司：知道模型架构往哪走，才知道芯片怎么设计
- 创业公司：才知道到底怎么去 follow 需求
- 嘉宾还反问："博通怎么知道（模型）往哪一步走呢？"

13.3 中国会走出自己的路径吗

"我认为在有更多芯片的情况下、在更开放的模型开源的情况下，是会走出自己的路径的。

意味着我们跟他的芯片选择可能不完全一样，然后我们做出来的东西我也能完成我的目标，我也能实现更好的 **token** 成本——这些东西我觉得都是有可能实现的。"

但确定性的轨迹现在还没有：

"我觉得确定性的轨迹可能现在没有，大家都在这里面找平衡点。国产的芯片到底行不行呢？那大家就会去找这个破局点。"

华为的思路作为例证：

"华为前几天不是说了它那个定律吗？（原稿音译为"套定律"）就是在我芯片制程受到制约的时候，通过一些芯片架构、其他方面的设计，去做一个更强的系统出来——也是在想各种方法。在现在这个情况下，我相信中国的 AI 市场和人才是足够的。大家在这个情势下会走出自己的路，或者会想出方法。"

13.4 百度的历史禀赋：昆仑芯

"我们的昆仑芯应该是 09 年还是 10 年就开始做了。当然最早没有做芯片，是做 FPGA 的。我想说的是：从那个时候我们的思路就是一下子到硬件去想办法，深入到底层。当时去优化的是搜索的 workload，只是说今天从搜索的 workload 变成了 AI 的 workload。相对其他一些公司来说，可能在这方面就有更多能够发挥的空间。"

14. 百度的打法与新指标 DAA

14.1 两个发力点

第一点：更高效的算力效率、token 效率

"大家都想往下做，但不代表你能把它做好，或者说你能在正确的时间点把它做好。这里面其实还是有很多（空间），我们会在这个上面去发力。"

第二点：靠云的生态，与客户一起长大

"现在云是我们一个很重要的业务出口。希望这一套有更多的客户帮我们一起把生态做得更好，大家一起去做下一代真正爆款的应用。包括现在的 agent 客户、具身智能的客户，我们希望在他们比较小的时候就跟我们合作。如果是在我们的云上面长出来的，那我们这套东西可能就更有竞争力。"

台积电的类比：

"台积电当年也是个小公司，就是因为某一单、跟某一家做了一款产品之后，发展到今天的——就在那个还比较小的时候（就合作了）。"

主持人的总结："和未来的（客户）走在一起。"

14.2 新指标：DAA（Daily Active Agents）

背景：硅谷很多公司（Groq、Baseten 等）都在重新定义 infra 指标。百度大会上 Robin（李彦宏）提出了一个新指标 DAA — Daily Active Agents。

百度内部的两级指标体系：

层级	指标	作用
底层（明确）	Token 效率 指标	"一个很明确的牵引，牵引我们整个往下走"——即"冰山下面那一大坨怎么做"的指引
上层（不那么明确）	DAA	"给我们一个往上衡量任务的指引"

为什么需要 DAA：

"前一段时间大家都在用 token 衡量，大家会发现 token 很快就被玩坏了。我拿 token 干一些没用的事儿，那这个 token 消耗没有价值。

所以 DAA 这个指标，我认为是帮我们指引"它真实完成的任务"以及"任务产生的价值"的一个指标。"

14.3 DAA 的价值计算逻辑与由此推导出的工作项

你能够长时间运行这个任务 → 代表这个任务价值是多的

↓

任务价值 × 数量 = 你最终创造的价值总量

↓

怎么让更多 agent 被开发出来？

- ├ 第一步：让人家开发得足够快
- ├ 第二步：让它运行得比较顺利，不要老出错
- ├ 出错了也可以自动恢复
- └ 结果也可以 check

还要区分任务价值高低：

"我们也会分任务的难易、价值高低。价值高的我就能花更多的功夫去做。不能只是为了数量去做——得把高价值的东西做到 DAA 里面来，变成 daily active 的 agent。"

成本路径：

"第一步我先从高价值的任务能跑到可用。如果可用了，我再想办法让他用更多的模型、更低成本的模型来做它。"

15. 为什么 Token 指标会失效

这一段是本期最坦诚的一段。

主持人的尖锐提问：

"云厂商在过去相当长一段时间，肯定是希望你消耗越多 token 越好——因为我就是水电公司，你消耗越海量 token 我就赚钱。但在你们的指标里，你不再用 token 的量去绝对衡量，而是更衡量这个 token 到底有没有被用好。这是一个转变吗？"

嘉宾的回答：

"首先，如果无脑地大家都用 **token**，那我们肯定很开心。

但这只是一个梦想。当大家都发现你的 **token** 用得不好的时候，谁还会来真的持续地用这个 **token**？"

关于 **harness** 为什么重要（成本控制视角）：

主持人："为什么现在要强调 **harness**？是因为你要是不去界定他的话，他可能一下子给你上了最多的账单。"

嘉宾："太可怕了。"

核心表态：

"所以你不能做杀鸡取卵的事，不能利用大家的恐慌心理，你就都来烧 **token**。这个东西肯定是不可持续的，我们也清楚地知道这一点，还是要想办法让大家真正地能够创造价值。"

15.1 一个已经被证伪的历史路径：行业微调专家模型

"这个东西已经变了好几轮了。回顾一下：大模型刚出来的时候，我们当时说要**赋能千行百业**，做法其实是**模型微调、专家模型**。

百度云当时投入了大量精力去帮人做专家模型，因为当时觉得大模型的能力不够，所以必须要把各个行业自己的知识往里面加。

但今天看，其实那条路已经行不通了。"

不变的追求："我们希望提供一个平台、一个东西，能够帮助大家更蓬勃地把应用做起来。至于产品功能在哪里、分界面在哪里，还在一个变化的过程中。"

16. 结语：为什么这是工程师最好的时代

16.1 Infra 人的兴奋点：从"刻板的效率提升"到"开放式探索"

"做 **infra** 的同学来讲，看到了这个变化、看到了你有很多可选项，其实就很兴奋了。

- **CPU 时代**：你是不会在芯片层面去想什么工作的
- **AI 时代**：你会发现我可以做很多系统层面的东西
- **Agent 时代**：我要做更不一样的系统——**agent OS** 这么一层抽象的概念，对于做系统的人来说都是值得兴奋的

你能探索的空间变得比原来大很多，你能做的选择和判断多很多了，而且影响更加大了，意味着你的束缚变少了。"

主持人的概括："现在是 **infra** 的探险乐园时代。"

"**Agent infra** 之后，**infra** 的范围其实又变得更大了，你往上往下其实都会走得更深入。"

16.2 更大的判断：整个研究范式都在工程化

这是全场的收尾金句。

"往大了一点讲，现在整个的研究范式都在工程化。现在即使是做一个算法，你也是用工程化的方式在做算法的。

你看大家都是 **Transformer** 架构，其实大家拼的不是灵光一闪的那个逻辑了。

你要把算法做好、要把 **infra** 做好、要把 **agent** 做好，你都要搭建一套工程架构，你都要有一套体系来驱动你这个工作能够更好地完成。

我们传统上大家分做算法的人、做工程的人——对于做工程的人，我觉得都是一个值得兴奋的时代。"

主持人："因为今天你用工程的方法去解决更复杂的问题。"

嘉宾：

"更多、更广泛的问题。对做工程的同学来说，我觉得都是一个更好的时代。"

16.3 关于最终格局的悬念

"现在大家笃定的就是：这一轮 **AI** 确实是一个生产力的革命。但是最后的局势到底是一家把这个东西全干了，还是剩下几个寡头——这个东西我也不好说。"

真的有可能一家全干了吗？

"我也不好说，这个不排除。就好比我说做代工的，那就是被台积电一家做。其他家虽然有三星、**Intel** 什么的，但其实相对来说都比较少。"

17. 术语勘误表

原始文稿是语音识别产物，术语错误密集。整理时已还原，此处备查。

文稿中的写法	实际术语
英法 / 音法 / 云法 / 引发 / A键的英法 / infrared / instance层 / infront / inflight	Infra （基础设施）
王彦鹏	王雁鹏
111 年	2011 年
小龙虾 / open call / Open class / open cloud	OpenClaw （其标志为龙虾）
俄罗斯 / armas / Hermas	（OpenClaw 之后爆火的另一 agent 产品，具体名称存疑）
honeys / harness / tennis	Harness
路虎 / loop	Loop （主循环）
minus	Manus
reg / rag	RAG
4.5 + cloud code	Claude 4.5 + Claude Code
SO pic / etho pic / F pic / s topic	Anthropic
心魔一体 / 星云模一体 / 新膜一体	芯云模一体（芯片-云-模型）
拍拓奇	PyTorch
jx	JAX
jm 李三 / 经理三 / 金 mini 3	Gemini 3
grog / group / brock / rock	Groq
哭大	CUDA
蓝圈	LangChain
brother base	Baseten
毛 TA	Multi-Agent
星际之门	Stargate
GPT s	GPT Store

文稿中的写法	实际术语
LPGA	FPGA
聚生智能	具身智能
彭博的临界点	蓬勃（发展）的临界点
专业的逗号	专业的门道
网上衡量任务	往上衡量任务
自动净化 skill	自动提炼 skill
DNA / EAA / DA infer	DAA (Daily Active Agents)
Robin n	Robin Li (李彦宏)
套定律（华为）	华为提出的某个定律，名称存疑
s logic	ASIC 设计相关公司，名称存疑
python（记忆存储公司）	Agent memory 基建创业公司，名称存疑
就聚焦做客户	就聚焦做 coding
CPU 的小	CPU 的消耗

18. 核心结论速查

1. **Infra 的定义**：抽象出一层，对下有很好的抽象，对上有很好的支撑，且影响面足够大。
2. **这一代 AI Infra 最大的变化**：从纯软件推向**软硬一体**。GPU 释放了算力但牺牲了通用性，做 AI Infra 的人必须从芯片体系结构做起。
3. **为什么 Infra 这轮变重要**：互联网时代 IT 是**成本项、无边际成本**；AI 时代**智能=算力**，算力是生产资料。越往下走越重资产（软件→模型→芯片→流片→电力）。
4. **Agent 三阶段**：Chat → 完成任务 → **进入生产环节**。现在卡在第三阶段前，**Demo 容易、线上系统难；完成任务容易、替代岗位难**。
5. **真正的制约是模型能力边界**：模型能力边界在哪里，决定了 infra 抽象层在哪里。这已经"杀死"了 prompt 工程那一波，正在考验 context engineering 这一波。
6. **Harness = Loop + Tools + Memory**。Loop 通用到能覆盖 90% 任务；工具层的关键选择是**文件系统而非 RAG**（RAG 是数据库、范式固定；文件系统是笔记本、想存什么存什么）。
7. **Claude Code 定义了范式，OpenClaw 把它泛化到全场景**（有记忆、有情绪价值、不需要用户有上下文）。

8. 模型会成为 OS，但 agent 会越来越多。模型吸收的是思考方式和工作范式；每项具体工作的 know-how、workflow、记忆、数据不会被完全吸收。
9. Agent 对 Infra 的核心冲击是上下文长度爆炸（已到 1M）。算力要多芯异构，存储是最紧的瓶颈（扩产要 2 年、HBM 短缺、行业刚从下行周期出来），网络反而变化不大（超节点有没有 agent 都会做）。
10. CPU 消耗量是判断 agent 是否真正落地的领先指标——GPU 思考、CPU 执行工具，CPU 消耗大涨 = agent 真在干活而不是“老思考”。
11. Agent Infra 三大件：运行时/沙箱（快速启停 + 安全边界）、编排系统（稳定高效可复现）、记忆系统（这一层的范式还没定，是最大的未知）。
12. “为自己做的 infra” ≠ “为所有人做的 infra”：TPU 给外部创业者用可能浪费 50%~80% 算力，因为它不是为你做的。这是训练侧生态难以撼动 NVIDIA 的根本原因。
13. 垂直整合是所有巨头的方向，但芯片 2.5 年的周期使它高度不确定——你今天用模型定义的芯片，两年半后大概率已经不是那个模型的需求了。
14. 推理系统是生态里唯一的缝隙：模型已经训好了，你有时间去做专门 for 推理的芯片。NVIDIA 收购 Groq 也是在防这一手。
15. 中国路径的两个差异：被迫的多芯异构（挑战），以及开源领先（结构性优势——开源让模型架构演进透明，芯片设计者才知道往哪走）。
16. Token 指标正在失效：token 很容易被“玩坏”（拿 token 干没价值的事）。云厂商不能利用恐慌心理让大家烧 token，那不可持续。DAA（Daily Active Agents）是往价值侧的校正。
17. 一条已被证伪的路：大模型早期“赋能千行百业 = 行业微调专家模型”，今天看这条路已经行不通了。
18. 整个研究范式都在工程化：大家都是 Transformer，拼的不再是灵光一闪，而是能不能搭起一套工程体系。这是做工程的人最好的时代。